

labwork 4

October 1, 2026

0.1 grayscale 2D

First, we want to create a new grayscale function that takes in input a 2 dimensional array. The input and output data are defined the same way as for the 1D grayscale:

```
devSrc2D = cuda.to_device(src)
devDst2D = cuda.device_array((height, width, 3), np.uint8)
```

This time, the block size is a tuple of the same value. Same for the grid size, we use the same principle as for the 1D but we do it with each dimensions.

```
blockSize2D = (value, value)
gridSize2D = ((width + value - 1) // value, (height + value - 1) // value)
```

For the grayscale2D this time the threads are represented as a 2D array so we define x and y. We check that there are not bigger than the size of the source otherwise it would raise an error.

```
@cuda.jit
def grayscale2D(src, dst):
    x = cuda.threadIdx.x + cuda.blockIdx.x * cuda.blockDim.x
    y = cuda.threadIdx.y + cuda.blockIdx.y * cuda.blockDim.y
    if y > src.shape[0] or x > src.shape[1]:
        return
    g = np.uint8((src[y, x, 0] + src[y, x, 1] + src[y, x, 2]) / 3)
    dst[y, x, 0] = dst[y, x, 1] = dst[y, x, 2] = g
```

0.2 Blocksize benchmark

For grayscale 1D and 2D we test different block sizes. To test correctly for each block size we run the grayscale function 50 times and get the average. A first grayscale function is run without taking its time to take only the computation time and not the compilation time. The destination is reinitialize each time we change the blocksize so that if there is an error it doesn't keep the old results but only the last one.

```
blockSizes1D = [4, 8, 16, 32, 64, 128, 256, 512, 1024]
blockSizes2D = [2, 4, 8, 16, 32]

for value in blockSizes1D:
    gridSize1D = (pixelCount + value - 1) // value
    total = 0
    devDst1D = cuda.to_device(np.zeros((pixelCount, 3), np.uint8))
    grayscale1D[gridSize1D, value](devSrc1D, devDst1D)
    for i in range(nbIter):
        start = time.time()
        grayscale1D[gridSize1D, value](devSrc1D, devDst1D)
        cuda.synchronize()
        stop = time.time()
        total += stop - start

    results1D.append(total / nbIter)

for value in blockSizes2D:
    blockSize2D = (value, value)
    gridSize2D = ((width + value - 1) // value, (height + value - 1) // value)
    total = 0
    devDst2D = cuda.to_device(np.zeros((height, width, 3), np.uint8))
    grayscale2D[gridSize2D, blockSize2D](devSrc2D, devDst2D)
    for i in range(nbIter):
        start = time.time()
        grayscale2D[gridSize2D, blockSize2D](devSrc2D, devDst2D)
        cuda.synchronize()
        stop = time.time()
        total += stop - start

    results2D.append(total / nbIter)
```

0.3 Results

As we can see, the best block size for 2D grayscale is (8,8). If we take the best time for 1D and for 2D and compare them we see that the difference is really close sometime 2D is faster sometimes slower.

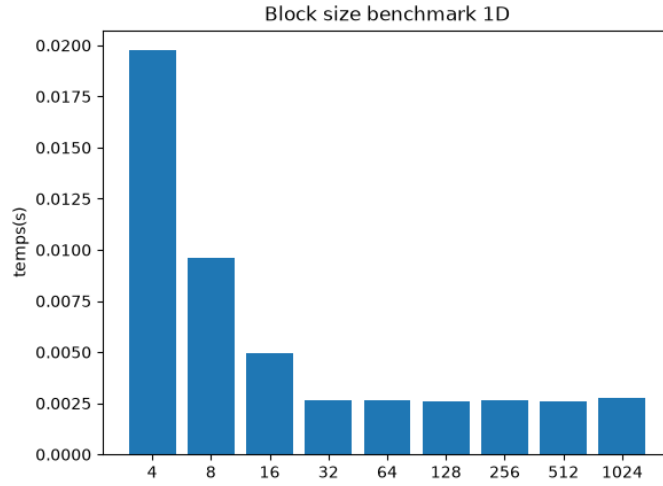


Figure 1: Block size benchmark 1D

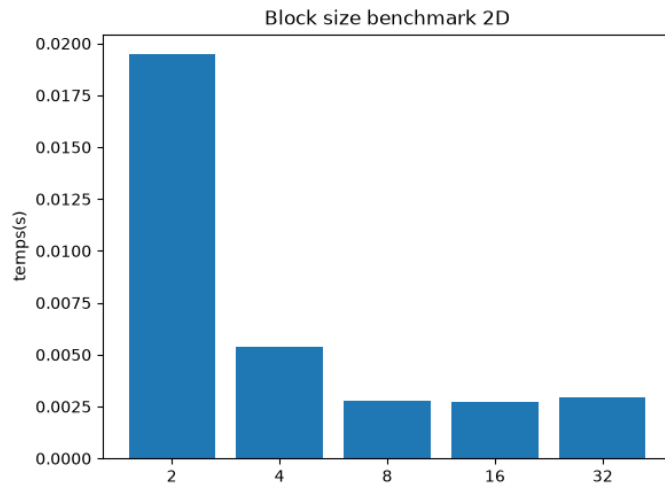


Figure 2: Block size benchmark 2D

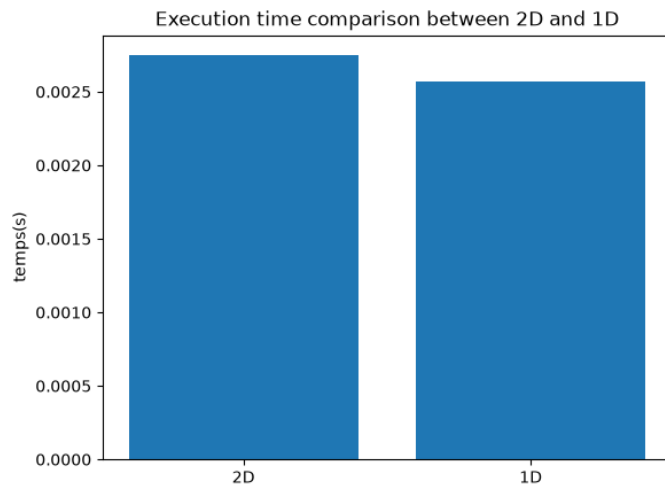


Figure 3: Execution time comparison between 2D and 1D